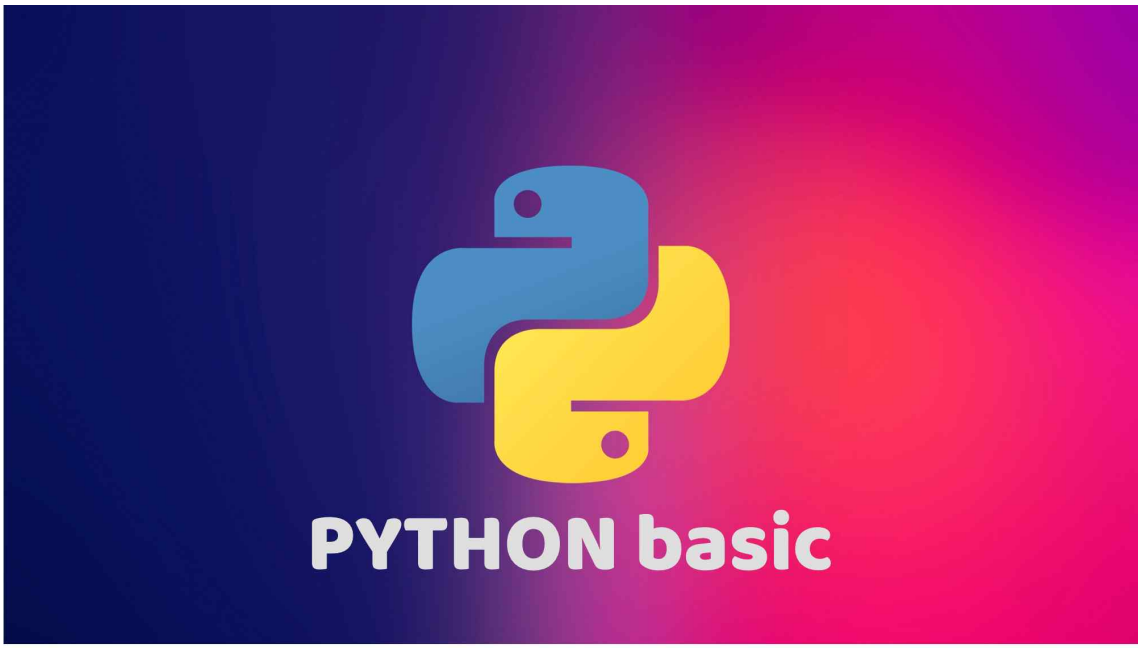
매우 중요한 기초 자료임 - 완전 숙지해야 프로그래밍이 쉬워짐

https://velog.io/@jewon119/01.-Python-%EA%B8%B0%EC%B4%88-%EB%8D%B0%EC%9D%B4%ED%84%B0-%ED%83%80%EC%9E%85data-type

1. 01. Python 기초- 데이터 타입(data type)

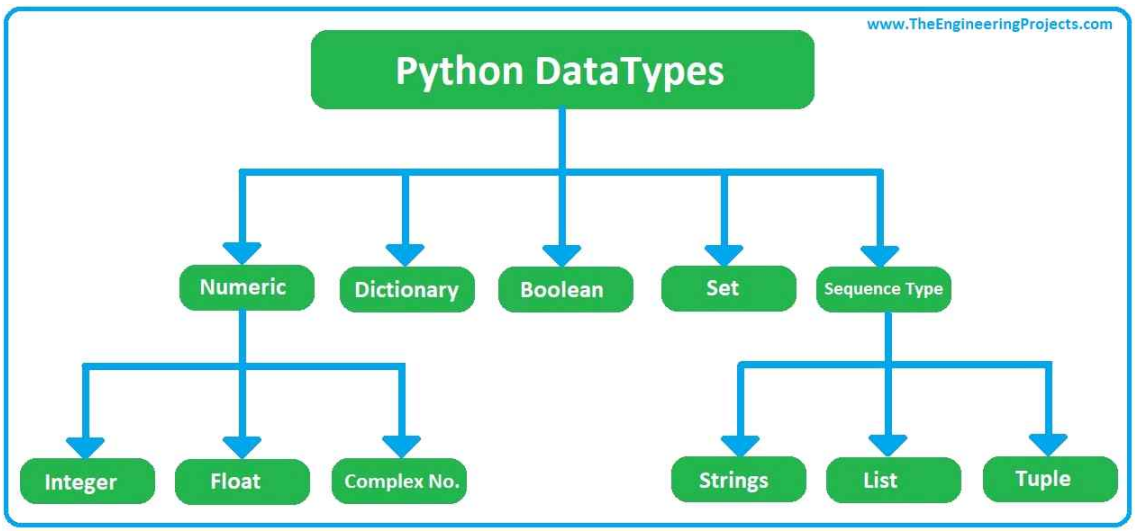
jewon119·2021년 2월 4일
python
5
□ 데이터 타입(data-type)
□ python은 6가지 데이터 타입이 존재
1. Numeric Types : Int / Float / Complex
2. Sequence Types : Str / List / Tuple
3. Mapping Type : dict
4. Set Type : set
5. Boolean Type : bool
□ 형변환(type change)
가. Python
목록 보기
1/39



나. □ 데이터 타입(data-type)

1) □ python은 6가지 데이터 타입이 존재

- 🕐 Numeric Types: int(정수), float(소수), complex(복소수)
- 🕐 Sequence Types: str(문자열), list(리스트), tuple(튜플)
- 🕐 Mapping Type: dict(딕셔너리)
- 🕐 Set Types: set(집합)
- 🕐 Boolean Type: bool(불리언)
- 🕐 Binary Types: bytes, bytearray, memoryview



✍□ pyrhon

```
# 데이터 타입
v_str1 = "Niceman" #str
v_str2 = "Goodgirl" #str
v_bool = True #bool
```

```
v_float = 10.3 #float
v_int = 7 #int
v_complex = 3 + 3j
v_dict = {
    "name" : "Jang",
    "age" : 32,
    "city" : "seoul"
} #dict
v_list = [3,5,7] #list
v_tuple = 3,5,7 #tuple
v_set = {7,8,9} #set
#
print(type(v_str1)) # <class 'str'>
print(type(v_str2)) # <class 'str'>
print(type(v_bool)) # <class 'bool'>
print(type(v_float)) # <class 'float'>
print(type(v_int)) # <class 'int'>
print(type(v_complex)) # <class 'complex'>
print(type(v_dict)) # <class 'dict'>
print(type(v_list)) # <class 'list'>
print(type(v_tuple)) # <class 'tuple'>
print(type(v_set)) # <class 'set'>
```

2) 1. Numeric Types : Int / Float / Complex

- Int(정수형) : 숫자형의 하나로 소수점이 없는 정수
- Float(소수형) : 숫자형의 하나로 소수점이 있는 실수
- Complex(복소수) : 실수와 허수의 합인 복소수
- 일반적으로 수학에서 가능한 모든 기능을 사용할 수 있음

🕐 import math를 통해 수학적 기능을 확장시켜 활용 가능

✍☐ python

```
# 사칙연산
# 덧셈(+), 뺄샘(-), 곱셈(*), 나눗샘(/), 몫(//), 나머지(%), 제곱(**)
a = 10
b = 2
print(a+b) # 12
print(a-b) # 8
print(a*b) # 20
print(a/b) # 5.0
print(a//b) # 5
print(a%b) # 0
print(a**b) # 100
#단항 연산자
x = 100
x += 100    # x = x + x
print(x) #200
y = 100
y /= y    # y = y / y
print(y) #1.0
z = 7
z *= z    # z = z * z
print(z) # 49
m = 99999999
m -= m    # m = m - m
print(m) # 0
# 수치 연산 함수
print(abs(-7)) #절대값 : 7
n, m = divmod(100,8)
print(n, m) #12 4
import math
```

```
print(math.ceil(5.1)) #올림 : 6
print(math.floor(3.874)) #내림 : 3
```

3) 2. Sequence Types : Str / List / Tuple

가) 1) 문자형(str)

🕐 문자열(String)이란 문자, 단어 등으로 구성된 문자들의 집합을 의미함

✍☐ python

```
# 문자열 만들기 1 : 큰따옴표(")로 양쪽 둘러싸기
print("Hello world!") # Hello python!
# 문자열 만들기 2 : 작은따옴표(')로 양쪽 둘러싸기
print('Hello world!') # Hello python!
# 문자열 만들기 3 : 큰따옴표 3 개를 연속("""")으로 써서 양쪽 둘러싸기
print""""Hello world!""") # Hello python!
# 문자열 만들기 4 : 작음따옴표 3 개를 연속(''')
print'''Hello world!''') # Hello python!
# 문자열에 '를 포함시키고 싶을 때 : 큰따옴표("")로 묶어줌
print("Python's is favorite food is pizza") # Python's is favorite food is pizza
# 문자열에 "를 포함시키고 싶을 때 : 작음따옴표('')로 묶어줌
print('"Hi, Dave" he says.') # "Hi, Dave" he says.
# 문자열에 큰따옴표(")나 작은따옴표(')를 포함시키고 싶을 때 : 왼쪽에 \(역슬래쉬)
food = 'Python\'s favorite food is perl'
print(food) # Python's favorite food is perl
say = "\"Python is very easy.\" he says."
print(say) # "Python is very easy." he says.
# 줄 바꿈 위한 이스케이프코드(\n) 삽입하기
multiline = "Life is too short\nYou need python."
print(multiline)
# Life is too short
# You need python.
```

- 🕐 문자형은 문자를 다룰 수 있는 다양한 추가, 삽입, 수정, 삭제 기능을 제공함

- 🕐 길이 구하기, 인덱싱, 슬라이싱, 갯수 세기, 위치 찾기, 삽입, 대체하기, 나누기 등..

✍☐ python

```
# 문자열 길이 구하기 : len()
getlength = 'python is cool'
print(len(getlength)) # 14
# 문자열 인덱싱
indexing = "Life is too short, You need Python"
print(indexing[3]) # e -> 0 번부터 시작, 3 은 4 번째 자리
print(indexing[-1]) # n -> -1 은 뒤에서부터 시작해 1 번째
# 문자열 슬라이싱
slicing = "Life is too short, You need Python"
print(slicing[0:4]) # Life -> [x:y] x 부터 y 전까지
문자열을 뽑아옴(끝 번호 포함 안함)
print(slicing[:]) # Life is too short, You need Python
print(slicing[:4]) # Life
print(slicing[18:]) # You need Python
print(slicing[-1:7]) # Python
# 문자 갯수 세기(count)
count_str = "lalalalalballlballballballlblblblblaaaa"
print(count_str.count("l")) # 20 -> count_str 중 l 의
갯수를 가져옴
# 위치 알려주기(find)
find_str = "python!"
print(find_str.find("!")) # 6 -> !표가 6 번째 자리에
위치함
print(find_str.find("a")) # -1 -> a 가 find_str 안에 없음
# 문자열 삽입(join)
join_str = "python"
```

```
join_str = "python"
print(".".join(join_str)) # p.y.t.h.o.n
print("-".join(join_str)) # p-y-t-h-o-n
# 소문자를 대문자로 바꾸기(upper)
a = "hi"
print(a.upper()) # HI
# 대문자를 소문자로 바꾸기(lower) #python
a = "PYTHON"
print(a.lower())
# 공백지우기 : 양쪽 공백 지우기(strip) / 왼쪽 공백
지우기(lstrip) / 오른쪽 공백 지우기(rstrip)
a = "    hi    "
print(a.strip()) # hi
# 문자열 바꾸기(replace)
a = "Life is too short"
print(a.replace("short", "long")) # Life is too long
# 문자열 나누기(split) : ()안에 있는 문자열을 기준으로해서
문자열을 나눈 후 나눠진 값을 리스트 넣어줌
a = "Life is too short"
print(a.split()) # ['Life', 'is', 'too', 'short']
b = "a:b:c:d"
print(b.split(":")) # ['a', 'b', 'c', 'd']
```

나) 2) 리스트(list)

- 리스트를 사용하여 데이터를 묶어서 사용할 수 있음

- 불필요하게 각 각의 변수를 생성하여 데이터를 하나씩 담아줘야하는 문제점을 개선

- 리스트는 문자형처럼 인덱싱(indexing)과슬라이싱(slicing) 사용 가능

- 또한, 수정, 삭제, 추가, 정렬, 뒤집기, 삽입 등 모두 가능

✍☐ python

```
# indexing : 리스트내에서 위치값으로 원하는 요소 추출하기
indexing = [1,2,3,4,5]
print(indexing) # [1,2,3,4,5]
print(indexing[0]) # 1
print(indexing[0]+indexing[-1]) # 6 => 1+5
indexing = [1,2,3, ["a","b","c"]]
print(indexing[0]) # 1
print(indexing[3]) # ["a", "b", "c"]
print(indexing[3][-1]) # c
# slicing : 리스트내에서 위치값으로 원하는 가져오기
slicing = [1, 2, 3, ["a", "b", "c"], 4, 5]
print(slicing[2:5]) # [3, ['a', 'b', 'c'], 4] ->
두번째부터 다섯번째 전까지 가져옴(마지막 숫자 포함안함)
# 리스트 값 수정하기
a = [1,2,3]
a[2] = 4 # 2 번째 자리를 4 로 바꿈
print(a) # [1, 2, 4]
# 리스트 값 삭제하기(del)
a = [1,2,3,4,5,6,7,8,9,10]
del a[9] # 9 번째 자리수 삭제
print(a) # [1,2,3,4,5,6,7,9]
del a[3:] # 3 번째자리수 이후 모두 삭제
print(a) # [1, 2, 3]
# 리스트 값 추가하기(append, extend)
a = [1,2,3,4,5]
a.append(6) # 리시트 a 에 요소로 6 을 넣음
print(a) # [1, 2, 3, 4, 5, 6]
a.append([7,8,9,10]) # 리스트 a 에 [7,8,9,10] 리스트를
내부리스트로 넣어줌
print(a) # [1, 2, 3, 4, 5, 6, [7, 8, 9, 10]]
b = [1,2,3,4,5]
b.extend(["a","b","c","d"]) # append 는 배열안에 배열로
추가하지만, extend 는 확장시킴
```

```
print(b) # [1, 2, 3, 4, 5, 'a', 'b', 'c', 'd']
# 리스트 정렬(sort)
a = [1,4,3,2,5]
a.sort() # 오른차순 정렬
print(a) # [1, 2, 3, 4, 5]
# 리스트 뒤집기(reverse)
a = ['a', 'b', 'c', 'z', 'f', 'i']
a.reverse()
print(a) # ['i', 'f', 'z', 'c', 'b', 'a']
# 리스트 요소 삽입(insert)
a = [1,2,3]
a.insert(0,10) # 0 번째 자리에 10 을 넣어줌
print(a) # [10, 1, 2, 3]
# 리스트 요소 제거(remove) : 첫번째로 나오는 요소를
삭제해줌
a = [1,2,3,1,2,3]
a.remove(3) # 리스트에서 첫번째로나오는 3 을 삭제
print(a) # [1, 2, 1, 2, 3]
# 리스트 요소 끄집어 내기(pop) - 마지막 요소를 꺼내주고 기존
리스트에서 해당 요소를 삭제
a = [1,2,3]
print(a) # [1,2,3]
a.pop() # 맨 뒤에 요소 1 개(3)를 끄집어 냄
print(a) # [1,2]
a.pop() # 맨 뒤에 요소 1 개(2)를 끄집어 냄
print(a) # [1]
a.pop() # 맨 뒤에 요소 1 개(1)를 끄집어 냄
print(a) # []
# 리스트에 포함된 요소 x 의 개수 세기(count)
a = [1,2,3,1]
b = ["a", "b", "c", "a", "d", "e", "a"]
print(a.count(1)) # 2 -> a 리스트에 1 이 2 개 있음
print(b.count("a")) # 3 -> b 리스트에 a 가 3 개 있음
```

다) 3) 튜플(tuple)

- 튜플은 2 가지 점을 제외하고는 전반적으로 리스트와 유사함

- 차이점 1. 리스트는 []로 감싸지만, 튜플은 ()로 감쌈

- 차이점 2. 리스트는 요소의 생성, 삭제, 수정이 가능하지만 튜플은 그값을 바꿀 수없음

- 다만, 튜플은 인덱싱, 슬라이싱, 더하기, 곱하기, 길이구하기 처럼 기본적인 것은 가능

- 덧셈은 튜플끼리도 가능하지만, 곱셈은 튜플에 숫자를 곱해서만 가능(튜플끼리 곱셉 안됨)

- 또한 튜플이 1 개의 값을 갖고 있다면 값뒤에 콤마(,)를찍어줘야하고, ()를 생략해도 가능

✍☐ python

```python
# 튜플 기본
tuple1 = ()
tuple2 = (1,)
tuple3 = (1,2,3)
tuple4 = 1,2,3
tuple5 = ("a", "b", ("ab", "cd", ("xyz",)))
print(tuple1) # ()
print(tuple2) # (1,)
print(tuple3) # (1, 2, 3)
print(tuple4) # (1, 2, 3)
print(tuple5) # ('a', 'b', ('ab', 'cd', ('xyz',)))
```

```
# 튜플 다루기
# 튜플 인덱싱하기
tuple5 = ("a", "b", ("ab", "cd", ("xyz",)))
print(tuple5[2]) # ('ab', 'cd', ('xyz',))
# 튜플 슬라이싱하기
tuple5 = ("a", "b", ("ab", "cd", ("xyz",)))
print(tuple5[1:]) # ('b', ('ab', 'cd', ('xyz',)))
# 튜플 더하기
tuple3 = (1,2,3)
tuple4 = 1,2,3
print(tuple3 + tuple4) # (1, 2, 3, 1, 2, 3)
# 튜플 곱하기
tuple3 = (1,2,3)
print(tuple3 * 3) # (1, 2, 3, 1, 2, 3, 1, 2, 3)
# 튜플 길이 구하기(len)
tuple5 = ("a", "b", ("ab", "cd", ("xyz",)))
print(len(tuple5)) # 3 -> ("ab", "cd", ("xyz",)
print(len(tuple5[2][2])) # 1 -> ("xyz",)
# 튜플값 활용 : 변수값 스왑할 때!
# 다른 언어들은 두변수의 값을 서로 바꾸기 위해서는 임시변수
temp 등을 만들어 옮겨서 다시 넣어줘야함
# 튜플은 x,y = y,x 로 간단하게 가능함
x = 3
y = 9
x,y = y,x
print(x) # 9
print(y) # 33
```

4) 3. Mapping Type : dict

가) 1) 딕셔너리(dict)

🕐 자바스크립트에서 배열과 유사하기 key 와 value 를갖고 있음

- 🕐 리스트와 같이 순차적인 index 값을 요구하지 않고 key 값을 통해 value 를 얻음

- 🕐 {} 중괄호로 감싸서 만들 수 있음. {key1:value1, key2:value2, key3:value3, ...key(N):value(N)}

- 🕐 요소를 추가, 삭제, 값가져오기, 지우기 등...

✍☐ python

```
# 딕셔너리 기본 형태
dict1 = {"name":"jawon", "age":20,
"phone":"010-1111-2222"}
dict2 = {1:"hi"}
dict3 = {"a":[1,2,3]}
dict4 = {}
# 딕셔너리 쌍 추가하기 : 리스트 추가와 유사하나 []안에
키값을 넣음
dict1 = {1:'a'}
dict1[2] = "b" # 추가
print(dict1) # {1: 'a', 2: 'b'}
dict1["name"] = "jaewon"  # 추가
dict1["age"] = 20 # 추가
print(dict1) # {1: 'a', 2: 'b', 'name': 'jaewon', 'age':
20}
dict1[3] = [1,2,3,4]
print(dict1) # {1: 'a', 2: 'b', 'name': 'jaewon', 'age':
20, 3: [1, 2, 3, 4]}
# 딕셔너리 요소 삭제하기(del) : del dictName[key 값] 으로
지울 수 있고, key:value 쌍이 지워짐
dict2 = {"name":"jaewon", "age":"20",
"location":"seoul", 1:"a", "list_a":[1,2,3,4,5]}
del dict2["list_a"]
```

```
print(dict2) # {'name': 'jaewon', 'age': '20',
'location': 'seoul', 1: 'a'}
del dict2[1]
print(dict2) # {'name': 'jaewon', 'age': '20',
'location': 'seoul'}
# 딕셔너리에서 value 값 얻기
# 리스트, 튜플, 문자열은 요솟값을 얻고자 할 때 인덱싱이나
슬라이싱 기법을 사용하였으나,
# 딕셔너리에서는 key 를 사용해 value 를 구하는 방법을 사용함
# 왜냐하면 딕셔너리에는 index 순서가 없고, key 으로 index
하기때문
a = {1:"a", 2:"b", "name":"jaewon", "age":20,
"a":[1,2,3,4,5]}
print(a[1]) # a
print(a["name"]) # jaewon
print(a["age"]) # 20
print(a["a"]) # [1,2,3,4,5]
# get 을 통해 value 값 가져오기 -> 위 처럼 key 를 통해
value 를 호출할 수 있으나, 없는 key 를 호출하면 오류가
발생함
# get 을 통해 value 를 가져올 때 존재하지 않는 key 를
이용하면 None 를 돌려줌
print("---------------")
a = {'name':'pey', 'phone':'0119993323', 'birth':
'1118'}
print(a["name"]) # pey
print(a.get("name")) # pey
# print(a["location"]) # 오류발생
print(a.get("location")) # None 반환
print(a.get("location", "value is not")) # value is not
반환 -> 두번째에 미리 디폴트 값을 정해 없을때 호출할 수 있음
# key 값을 순차적으로 추출하기
c = {'name': 'pey', 'phone': '0119993323', 'birth':
'1118'}
for k in c.keys():
```

```
    print(k) # name, phone, birth -> 키값을 한개씩 불러올 수 있음
# key 값을 추출하여 리스트 만들기 : dictName.key()
list_c_keys = list(c.keys()) # 키값을 뽑은 후에 리스트로 감쌈
print(list_c_keys) # ['name', 'phone', 'birth']
# value 값을 추출하여 리스트 만들기 : dictName.values()
list_c_values = list(c.values())
print(list_c_values) # ['pey', '0119993323', '1118']
# key, value 쌍으로 리스트에 넣기 : dictName.items()
list_c_items = list(c.items())
print(list_c_items) # [('name', 'pey'), ('phone', '0119993323'), ('birth', '1118')]
# key 와 value 값 모두 지우기 : dictName.clear()
d = {"name":"jaewon", "age":20, "location":"seoul", "lulu":[1,2,3,4,5]}
d.clear()
print(d) # {}
# 해당 key 가 딕셔너리에안에 있는지 확인하기(in) -> 있으면 True, 없으면 False 돌려줌
a = {'name':'pey', 'phone':'0119993323', 'birth': '1118'}
print("name" in a) # True
print("birth" in a) # True
print("location" in a) # False
```

5) 4. Set Type : set

가) 1) 집합(set)

- 집합 자료형은 set 키워드를 사용해 만들 수 있음.
  비어있는 집합 : set()

- 집합은 중복을 허용하지 않기 때문에 집합으로 변환시켰을 때 중복된 값은 제거됨(종종 중복을 없애는 필터 기능으로 사용됨)

- 🕐 집합은 순서가 없기 때문에 저장된 값을 인덱싱으로 접근 불가

- 🕐 집합에서 인덱싱으로 저장된 값을 저장하기 위해서는 자료형을 리스트나 튜플로 변환 후 제어해야 함

- 🕐 집합은 교집합, 합집합, 차집합을 구하는데 이용함

✍☐ python

```
# set의 기본 형태 : 리스트형태나 문자열 형태로 만들 수 있음
s1 = set([1,2,3,1,2,3])
s2 = set("Hello")
print(s1) # {1,2,3}
print(s2) # {'l', 'o', 'e', 'H'} -> 중복이 제거됨
# 집합에서 인덱싱을 할 수 없기 때문에 요소값을 추출하려면 리스트나 튜플로 변환시킨 후 제어함
list_s1 = list(s1) # 리스트 변환
list_s2 = list(s2) # 리스트 변환
print(list_s1) # [1, 2, 3]
print(list_s2) # ['e', 'o', 'H', 'l']
print(list_s1[1]) # 2
print(list_s2[1]) # H -> 계속 값이 바뀜,, 리스트로 바뀌는 과정에서 순서가 랜덤으로 위치함
# 교집합, 합집합, 차집합 구하기
s1 = set([1, 2, 3, 4, 5, 6])
s2 = set([4, 5, 6, 7, 8, 9])
# 교집합 : 교집합은 집간 간 공통으로 있는 요소를 추출함
print(s1&s2) # {4, 5, 6}
print(s1.intersection(s2)) # {4, 5, 6}
# 합집합 : 중복없이 집간 간 요소들을 모두 합침
print(s1|s2) # {1, 2, 3, 4, 5, 6, 7, 8, 9}
print(s1.union(s2)) # {1, 2, 3, 4, 5, 6, 7, 8, 9}
```

```
# 차집합 : 집합 간의 뺄셈
print(s1-s2) # {1, 2, 3}
print(s1.difference(s2)) # {1, 2, 3}
# 집합 관련된 함수들
# 값 1개 추가하기(add) : 1개만 가능
s1 = set([1,2,3])
s1.add(4)
print(s1) # {1, 2, 3, 4}
# 값 여러개 추가하기(update)
s1 = set([1,2,3])
s1.update([4,5,6])
print(s1) # {1, 2, 3, 4, 5, 6}
# 특정 값 제거(remove)
s1 = set([1,2,3])
s1.remove(3)
print(s1) # {1, 2}
#  set 활용 : 중복없음, 집합 연산
data1 = {"apple", "samsung", "lg"}
data2 = {"samsung", "lg", "MS"}
print(data1&data2) # {'lg', 'samsung'} -> 교집합
print(data1|data2) # {'lg', 'MS', 'apple', 'samsung'}
->합집합
print(data1-data2) # {'apple'} -> 차집합
print(data1^data2) # {'MS', 'apple'} -> 합집합-교집합
#중복 없애기
data_list = ["apple", "samsung", "lg", "samsung", "lg",
"MS", "dell", "HP", "dell", "HP", "samsung"]
# {'dell', 'samsung', 'MS', 'lg', 'HP', 'apple'}
```

6) 5. Boolean Type : bool

🕐 불 자료형은 참(True)와 거짓(False)로 나타내는 자료형임. 이 2가지 값만 가질 수 있음

- 빈자료값("", (), {}, [] 등), 숫자 0, None 는 False, 나머지는 True

- 조건문과 함께 활용

다. □ 형변환(type change)

- type() 함수를 통해 형변환을 할 수 있음

✍□ python

```
int1 = 10
int2 = 200
float1 = .5
float2 = 10.
#형변환 int(정수), float(실수), complex(복소수)
print(int(float1)) #0
print(int(float2)) #10
print(float(int1)) #10.0
print(complex(3)) #(3+0j)
print(int(True)) #1
print(int(False)) #0
print(type(int(True))) #class 'int'
print(type(int(False))) #class 'int'
print(int('7')) # 7
print(complex(False)) #0j
```